

Exact Subset Sum: Reproducible Baselines, Measurements, and a Bounded-Magnitude Result

Reproducible Research Note

Abstract—We study the exact decision problem: given integers $a_1, \dots, a_n, b$, is there $x \in \{0, 1\}^n$ with $\sum_i a_i x_i = b$? Whether the classical $\mathcal{O}^*(2^{n/2})$ worst-case barrier can be improved to $\mathcal{O}^*(2^{n/3})$ is a long-standing open question; we do not resolve it. We instead collect deterministic witness-producing solvers—sorted meet-in-the-middle, a signed dynamic program, and Schroeppe–Shamir—give mathematical correctness and runtime arguments, formalize the correctness layer in Lean 4, and measure all three on infeasible instances. The measurements reproduce the predicted $\mathcal{O}^*(2^{n/2})$ time and confirm the $\mathcal{O}^*(2^{n/4})$ peak-memory advantage of Schroeppe–Shamir over meet-in-the-middle. A signed dynamic program solves every instance with magnitude sum $W = \sum_i |a_i| \leq 2^{\lfloor n/3 \rfloor}$ in $\mathcal{O}^*(2^{n/3})$ time; this is the textbook pseudopolynomial bound and not a general algorithm. The open bound, the restricted theorem, the proofs, and the finite measurements are kept strictly distinct.

Index Terms—subset sum, exact exponential algorithms, meet-in-the-middle, Schroeppe–Shamir, formal verification, reproducibility

I. PROBLEM AND SCOPE

Given $a_1, \dots, a_n, b \in \mathbb{Z}$, determine whether

$$\exists x \in \{0, 1\}^n : \sum_{i=1}^n a_i x_i = b. \quad (1)$$

The empty subset is permitted. Equal-valued entries are distinct occurrences, each available at most once. Negative entries and targets are permitted. This is the equality-decision form of 0–1 knapsack.

A classical result decides (1) on every input in $\mathcal{O}^*(2^{n/2})$ worst-case time, and Schroeppe–Shamir attain the same time with $\mathcal{O}^*(2^{n/4})$ space [1], [2]. Whether the exponent can be reduced to $1/3$ is open. Average-case and hard-knapsack improvements [3], [4] do not supply a worst-case guarantee, and the recent polynomial-factor improvement of Chen, Jin, Randolph, and Servedio [5] leaves the exponential base at $1/2$. Most recently, Randolph and Węgrzycki [7] break the meet-in-the-middle barrier in the worst case for several subset-balancing coefficient sets, but not for the $\{0, 1\}$ form (1). We therefore treat the $1/3$ exponent as open and make no claim about it.

We use $\mathcal{O}^*(\cdot)$ to suppress factors polynomial in n and in the input bit length. A bound polynomial only in n cannot hide the cost of reading arbitrarily long integers. Our algorithms use exact integer arithmetic; Python integers do not overflow. The runtime bounds below include polynomial bit costs rather than assuming arbitrarily large integers have unit cost.

II. ENUMERATION AND MEET-IN-THE-MIDDLE

For a list A , let $S(A)$ be the list of all subset sums, retaining multiplicities:

$$S([]) = [0], \quad (2)$$

$$S(a :: A) = S(A) ++ \{a + s : s \in S(A)\}. \quad (3)$$

Here $++$ denotes list concatenation. In Lean, the independent inductive proposition `HasSum` has constructors for selecting no elements, skipping the head, and taking the head.

Lemma 1 (Enumeration correctness). *$b \in S(A)$ if and only if A has a 0–1 selection summing to b . Moreover, $|S(A)| = 2^{|A|}$.*

Proof. For the empty list the only sum is zero. For a nonempty list, every selection either skips its head, yielding a sum in $S(A)$, or takes it, yielding $a + s$ for $s \in S(A)$. Conversely either construction produces a valid selection; induction proves both implications. The lengths obey $L_0 = 1$ and $L_{m+1} = 2L_m$, hence $L_m = 2^m$. Repeated sums do not affect this count because it counts list entries rather than distinct integers. $\square$

Lemma 2 (Split correctness). *For any lists L, R , a selection of $L ++ R$ sums to b if and only if there are $u \in S(L)$ and $v \in S(R)$ with $u + v = b$.*

Proof. Restrict a selection to the two disjoint ranges of occurrences; their sums are u and v . Conversely, concatenate two valid selections. No occurrence is reused because the ranges are disjoint. The argument assumes neither positivity nor distinct entry values. $\square$

Algorithm 1 sorts one half-sum list and binary-searches it for each sum of the other half. Attaching an occurrence bit mask to every sum yields a witness.

Theorem 1 (General baseline). *There is a deterministic exact witness-producing algorithm for (1) in time and space $\mathcal{O}^*(2^{n/2})$.*

Proof. Split after $k = \lfloor n/2 \rfloor$ entries. Enumerate the two half-sum lists with an occurrence bit mask per sum, sort the right list by sum, and for every left sum u binary-search for $b - u$. By Lemma 2 a match exists exactly when the original problem has a solution; concatenating the masks gives a witness. With $P = 2^k$ and $Q = 2^{n-k}$ entries, enumeration costs $\mathcal{O}(P + Q)$ arithmetic and mask operations, sorting $\mathcal{O}(Q \log Q)$ comparisons, and searching $\mathcal{O}(P \log(Q + 1))$ comparisons. Since both halves have size at most $\lceil n/2 \rceil$, the total is $\mathcal{O}(n2^{\lceil n/2 \rceil})$.

Algorithm 1 Sorted meet-in-the-middle

Require: entries $a_1, \dots, a_n$; target b **Ensure:** occurrence indices summing to b , or $\perp$

```

1:  $k \leftarrow \lfloor n/2 \rfloor$ 
2:  $L \leftarrow \text{ENUMERATE}(a_1, \dots, a_k)$ 
3:  $R \leftarrow \text{SORTBYSUM}(\text{ENUMERATE}(a_{k+1}, \dots, a_n))$ 
4: for  $(u, m_L) \in L$  do
5:    $p \leftarrow \text{LOWERBOUND}(R, b - u)$ 
6:   if  $p \leq |R|$  and  $R[p].\text{sum} = b - u$  then
7:     return the indices in  $m_L \mid (R[p].\text{mask} \ll k)$ 
8:   end if
9: end for
10: return  $\perp$ 

```

Algorithm 2 Dense signed-sum dynamic programming

Require: entries $a_1, \dots, a_n$; target b **Ensure:** occurrence indices summing to b , or $\perp$

```

1:  $W \leftarrow \sum_i |a_i|$ 
2: if  $|b| > W$  then
3:   return  $\perp$ 
4: end if
5:  $D \leftarrow [\perp] \times (2W + 1)$ ;  $D[W] \leftarrow 0$ 
   cell  $p$  represents sum  $p - W$ 
6: for  $i \leftarrow 1$  to  $n$  do
7:    $D' \leftarrow \text{copy of } D$ 
8:   for position  $p$  with  $D[p] \neq \perp$  do
9:      $q \leftarrow p + a_i$ 
10:    if  $0 \leq q \leq 2W$  and  $D'[q] = \perp$  then
11:       $D'[q] \leftarrow D[p] \mid (1 \ll (i - 1))$ 
12:    end if
13:  end for
14:   $D \leftarrow D'$ 
15: end for
16: return the indices in  $D[b + W]$ , or  $\perp$  if unreachable

```

operations and $\mathcal{O}(2^{\lceil n/2 \rceil})$ records. Each sum and mask has polynomial bit length in the input size, giving the stated $\mathcal{O}^*$ bounds. No probabilistic hashing is used. $\square$

III. SIGNED DYNAMIC PROGRAMMING

Let $W = \sum_i |a_i|$. All partial subset sums lie in $[-W, W]$. Maintain a table indexed by this interval, with either no witness or an occurrence mask per cell. Initially only the zero cell is reachable. For entry a_i build a new table from the old one by

$$D_i(s) = D_{i-1}(s) \vee D_{i-1}(s - a_i), \quad (4)$$

interpreting indices outside $[-W, W]$ as unreachable. Separate old and new buffers ensure an entry is never selected twice. Algorithm 2 gives the procedure.

Theorem 2 (Signed dynamic programming). *Algorithm 2 correctly decides (1) and returns a valid occurrence witness in $\mathcal{O}(n(2W + 1))$ cell visits.*

Proof. After processing i entries, the invariant is that cell s is populated exactly when some subset of the first i occurrences sums to s , and any stored mask describes such a subset. Initialization establishes the invariant at $i = 0$. Skipping a_i preserves old witnesses; taking a_i extends an old witness by a fresh occurrence bit and shifts its sum by a_i . These are precisely the cases of (4), so induction gives soundness and completeness. The triangle inequality keeps every true partial sum inside the table interval, and a target outside it is impossible. There are n scans of $2W + 1$ cells plus buffer copies of the same order. $\square$

IV. A RESTRICTED ONE-THIRD EXPONENT

Theorem 3 (Bounded-magnitude family). *On instances satisfying $W \leq 2^{\lceil n/3 \rceil}$, problem (1) can be solved in $\mathcal{O}^*(2^{n/3})$ time.*

Proof. Substitute the hypothesis in the cell-visit bound: $n(2W + 1) \leq n(2 \cdot 2^{\lceil n/3 \rceil} + 1)$. Witness masks have at most n bits; arithmetic, indexing, and copying add polynomial bit costs. Suppressing these factors gives the result. $\square$

This is the standard pseudopolynomial argument, not a new general exponential algorithm. For entries $a_i = 2^{i-1}$ we have $W = 2^n - 1$, so the hypothesis need not hold; such powers also yield 2^n distinct subset sums, showing that deduplicating sums alone does not force a better exponent. Splitting into three blocks does not repair this either: enumerating each block is cheap, but a naive merge of two block lists can contain $2^{2n/3}$ pairs. These observations describe failures of particular approaches, not a lower bound against all algorithms.

V. SCHROEPPEL-SHAMIR

The same $\mathcal{O}^*(2^{n/2})$ time is attainable with substantially less space by splitting into four blocks and streaming, rather than storing, the pair sums. Enumerate and sort the four block-sum lists ($2^{n/4}$ entries each). Stream the pairs of blocks 0+1 in nondecreasing order and the pairs of blocks 2+3 in nonincreasing order using a heap that merges $2^{n/4}$ sorted lists, then sweep the two streams toward each other with two pointers (Algorithm 3). Each stream advances at most $2^{n/2}$ times.

Theorem 4. *Algorithm 3 correctly decides (1) and returns a valid witness in $\mathcal{O}^*(2^{n/2})$ time and $\mathcal{O}^*(2^{n/4})$ space.*

Proof. By Lemma 2 applied twice, a solution exists exactly when some $u = u_0 + u_1$ with $u_0 \in S(G_0), u_1 \in S(G_1)$ and some $v = v_0 + v_1$ with $v_0 \in S(G_2), v_1 \in S(G_3)$ satisfy $u + v = b$. The heap merge yields U in nondecreasing order because, for fixed u_0 , the values $u_0 + S(G_1)$ are sorted; the symmetric argument gives V nonincreasing. The two-pointer sweep is standard: when $u + v < b$, no $v' \leq v$ can pair with the current u , so advance u ; when $u + v > b$, advance v . Hence the sweep finds a match iff one exists. The four sorted lists and the heap hold $\mathcal{O}(2^{n/4})$ entries. Both streams advance at most $2^{n/2}$ times and each heap step costs $\mathcal{O}(\log 2^{n/4}) = \mathcal{O}(n)$, giving $\mathcal{O}(n2^{n/2}) = \mathcal{O}^*(2^{n/2})$ time and $\mathcal{O}(2^{n/4})$ records. Occurrence

Algorithm 3 Schroeppe–Shamir with streamed pair sums**Require:** entries $a_1, \dots, a_n$; target b **Ensure:** occurrence indices summing to b , or $\perp$

```

1: partition the entries into four blocks  $G_0, \dots, G_3$  of size  $\lfloor n/4 \rfloor$ 
2: for  $t \leftarrow 0$  to 3 do
3:    $S_t \leftarrow \text{SORTBYSUM}(\text{ENUMERATESHIFTS}(G_t))$ 
4: end for
5:  $U \leftarrow \text{MERGEASCENDING}(S_0, S_1)$   $\triangleright$  streams  $u_0 + u_1$  increasing
6:  $V \leftarrow \text{MERGEDESCENDING}(S_2, S_3)$   $\triangleright$  streams  $v_0 + v_1$  decreasing
7:  $u \leftarrow \text{next}(U)$ ;  $v \leftarrow \text{next}(V)$ 
8: while  $u \neq \perp$  and  $v \neq \perp$  do
9:   if  $u.\text{sum} + v.\text{sum} = b$  then
10:    return the indices in  $u.\text{mask} \mid v.\text{mask}$ 
11:   else if  $u.\text{sum} + v.\text{sum} < b$  then
12:     $u \leftarrow \text{next}(U)$ 
13:   else
14:     $v \leftarrow \text{next}(V)$ 
15:   end if
16: end while
17: return  $\perp$ 

```

TABLE I

RUNTIME IN MILLISECONDS (BEST OF THE TIMED RUNS) ON INFEASIBLE INSTANCES; “–” MARKS A CONFIGURATION NOT RUN. BOUNDED VALUES ARE EVEN INTEGERS IN $[-100, 100]$, WIDE VALUES IN $[-2^{40}, 2^{40}]$; THE ODD TARGET FORCES EACH SOLVER TO RUN TO COMPLETION.

n	bounded values			wide values	
	DP	MITM	Schroeppe–Shamir	MITM	Schroeppe–Shamir
8	0.87	0.01	0.02	0.01	0.03
12	2.20	0.04	0.09	0.05	0.11
16	4.37	0.21	0.38	0.19	0.38
20	9.88	0.79	1.58	0.99	1.69
24	11.10	4.31	6.40	4.65	6.92
28	10.18	31.87	39.57	30.37	31.07
32	18.35	123.53	151.87	140.81	150.72
36	22.53	602.75	636.24	689.86	670.18

masks are carried with each sum, so a returned match yields a witness. $\square$

VI. MEASUREMENTS

We implement all four methods in one dependency-free Python module and measure them on deterministic infeasible instances. Each entry is an even random integer and the target is odd, so every solver must run to completion instead of stopping at an early match; this isolates worst-case behaviour. Two families are used: *bounded* values in $[-100, 100]$ (so the pseudopolynomial table fits) and *wide* values in $[-2^{40}, 2^{40}]$ (so the table does not). Times are the best of three runs; memory is the peak tracked by `tracemalloc`. The harness records provenance (commit, interpreter, platform, seed) alongside the CSV data.

Tables I and II and Figures 1–3 report the results.

TABLE II

PEAK TRACED MEMORY IN KIB ON BOUNDED-VALUE INSTANCES (LOWER IS BETTER). MITM GROWS LIKE $2^{n/2}$; SCHROEPPPEL–SHAMIR LIKE $2^{n/4}$.

n	DP	MITM	Schroeppe–Shamir
8	28.10	1.00	1.20
12	51.60	3.30	2.20
16	77.90	17.20	4.40
20	101.90	119.70	9.40
24	130.80	891.90	19.40
28	96.20	3855.40	35.90
32	136.40	15352.90	71.20

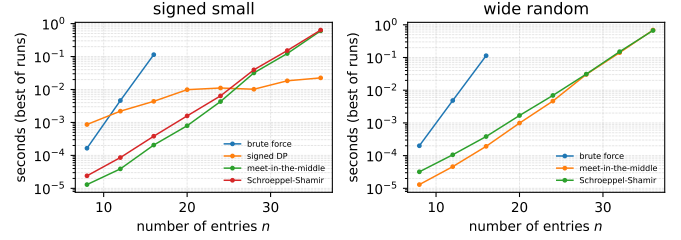


Fig. 1. Measured runtime versus n on infeasible instances, log scale. Wide values (right) admit no pseudopolynomial table, and meet-in-the-middle and Schroeppe–Shamir both grow exponentially; bounded values (left) are solved in near-constant time by the signed DP.

The measurements support three observations. First, meet-in-the-middle and Schroeppe–Shamir have the same exponential time order ($\mathcal{O}^*(2^{n/2})$); their measured times differ by a small constant factor from the heap operations. Second, the peak-memory curves separate cleanly, matching $2^{n/2}$ for meet-in-the-middle and $2^{n/4}$ for Schroeppe–Shamir. Third, the signed DP is effectively linear in n for bounded values and becomes the fastest method once n is large enough, but it is unusable on the wide family because W exceeds memory. Finite measurements illustrate the bounds; they do not prove an asymptotic improvement or establish any lower bound.

VII. FORMALIZATION AND EXECUTABLE ARTIFACTS

The file `lean/SubsetSum.lean` imports only `Std` under Lean 4.19.0. It proves:

- `mem_sums_iff`: enumeration is equivalent to the inductive selection specification;
- `sums_length`: the enumeration has 2^n entries;
- `hasSum_append`: the split equivalence;
- `meetInMiddle_correct`: executable list-based matching decides the inductive specification;
- `hasSum_magnitude` and `target_outside_impossible`: every feasible sum is inside the signed DP radius, and targets outside it are impossible;
- `restricted_work_bound`: the arithmetic inequality used in Theorem 3.

The Lean matcher deliberately uses linear list membership; it is an executable correctness reference, not the efficient sorted matcher. The formalization does not verify the Python

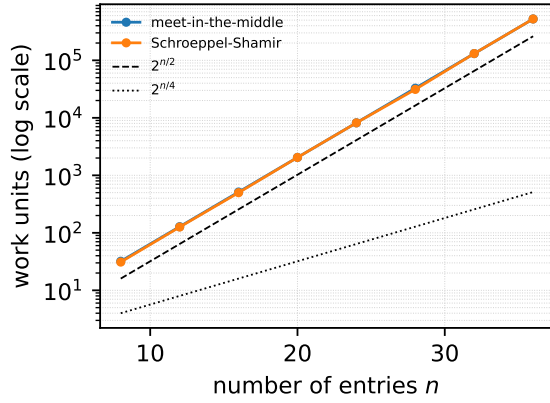


Fig. 2. Reported work units versus n (log scale) with the reference slopes $2^{n/2}$ and $2^{n/4}$. The measured counts track $2^{n/2}$ as predicted; the $2^{n/4}$ line bounds the space, not the time.

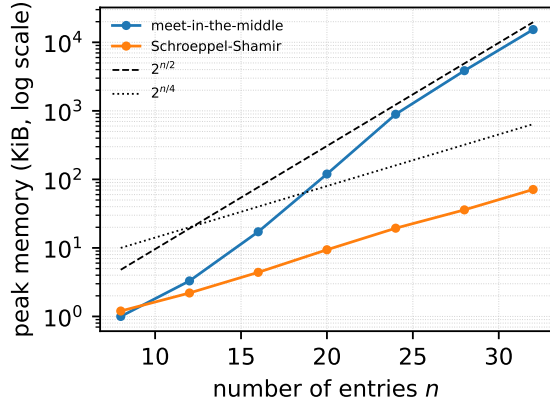


Fig. 3. Peak memory versus n (log scale). Meet-in-the-middle grows by about a factor of four per four added entries ($2^{n/2}$); Schroeppel–Shamir grows by about a factor of two ($2^{n/4}$), realizing the space saving its analysis predicts.

interpreter, the witness-mask implementation, the sorting refinement, the DP table, the Schroeppel–Shamir control flow, or operational running time. The runtime and DP correctness arguments are mathematical; the restricted Lean theorem is an arithmetic work envelope. The general $\mathcal{O}^*(2^{n/3})$ question is neither assumed as an axiom nor stated as a completed theorem. All proofs compile without `sorry`, and axiom reports are included in the source.

The Python module `src/subset_sum.py` has no third-party dependencies. The solvers return indices into the original sequence or `None` for infeasibility; the empty tuple is a valid witness. Automatic selection compares simple work estimates and makes no optimality claim. Schroeppel–Shamir is selected explicitly because its larger constant offset is not worth paying unless space matters.

VIII. REPRODUCTION AND VALIDATION

From the repository root:

```
python3 -m unittest tests.test_subset_sum -v
```

```
python3 -m src.subset_sum --values 3 -2 7 0 --target 5 --
method ss
python3 scripts/analysis/benchmark.py
uv run python scripts/analysis/plot_benchmarks.py #
figures + tables
cd lean && lake build
```

The unittest suite checks all lists of length zero through four over $\{-2, 0, 3\}$ against all targets from -9 through 13 (2,783 exhaustive instances), 150 seeded random instances with up to ten entries, explicit single-use and repeated-value cases, arbitrary-precision arithmetic, and input validation. Every solver is compared against independent Boolean-vector enumeration and each returned witness is checked for unique, in-range indices and the correct sum. Passing finite tests supports implementation confidence; it is not evidence for a general asymptotic bound. The benchmark writes CSV tables and a provenance record under `docs/analysis/2026-10-03/subset-sum/`; the figures and Tables I–II are regenerated from that data.

IX. REMAINING RESEARCH GOAL

To improve the worst-case bound, one must construct an algorithm with an $\mathcal{O}^*(2^{n/3})$ upper bound for *every* allowed input, justify its arithmetic model, and account for any randomized success guarantee. Neither favorable benchmarks nor average-case representation analyses supply that quantifier. The recent worst-case progress for subset-balancing coefficient sets [7] suggests the representation technique is the most promising direction; transferring it to the $\{0, 1\}$ form remains open. This note provides checked correctness foundations, a clean space-efficient baseline, and reproducible measurements; the unrestricted exponent is left open.

REFERENCES

- [1] E. Horowitz and S. Sahni, “Computing partitions with applications to the knapsack problem,” *Journal of the ACM*, vol. 21, no. 2, pp. 277–292, 1974.
- [2] R. Schroeppel and A. Shamir, “A $T = O(2^{n/2})$, $S = O(2^{n/4})$ algorithm for certain NP-complete problems,” *SIAM Journal on Computing*, vol. 10, no. 3, pp. 456–464, 1981.
- [3] N. Howgrave-Graham and A. Joux, “A new generic algorithm for hard knapsacks,” in *EUROCRYPT*, 2010.
- [4] A. Becker, J.-S. Coron, and A. Joux, “Improved generic algorithms for hard knapsacks,” in *EUROCRYPT*, 2011; IACR ePrint 2011/474.
- [5] X. Chen, Y. Jin, T. Randolph, and R. A. Servedio, “Subset sum in time $2^{n/2}/\text{poly}(n)$,” in *RANDOM*, 2023; arXiv:2301.07134.
- [6] J. Nederlof and K. Węgrzycki, “Improving Schroeppel and Shamir’s algorithm for subset sum via orthogonal vectors,” in *STOC*, 2021; arXiv:2010.08576.
- [7] T. Randolph and K. Węgrzycki, “Beating meet-in-the-middle for subset balancing problems,” arXiv:2511.10823, 2025.